

Exploring Orthogonality Constraints for Gaussian Process Additive Models

Ansh Sharma

June 2, 2025

Abstract

We investigate the impact of orthogonality constraints in Gaussian Process (GP) models on the decomposition of predictive functions into main effects and interaction terms. We compare two GP models: one enforcing same-feature orthogonality and another implementing cross-feature orthogonality. We analyse the covariance and variance of the decomposed components to understand how orthogonality assumptions influence model interpretability and performance.

1 Introduction

Modeling complex systems often requires capturing both the individual effects of features and their interactions. Gaussian Process (GP) models are a powerful non-parametric method for regression tasks, capable of modeling intricate patterns in data. Incorporating orthogonality constraints into GP kernels can influence how the model separates main effects from interaction terms. This report explores how same-feature and cross-feature orthogonality constraints affect the decomposition of GP models and the implications for interpretability and performance.

2 Theoretical Background

2.1 Gaussian Processes

A Gaussian Process is a collection of random variables, any finite number of which have a joint Gaussian distribution [1]. It is fully specified by its mean function $m(\mathbf{x})$ and covariance function $k(\mathbf{x}, \mathbf{x}')$:

$$f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')).$$

In regression tasks, GPs are used to infer the distribution over functions that fit the observed data, providing both predictions and uncertainty estimates.

2.2 Kernel Functions and Orthogonality

The kernel function $k(\mathbf{x}, \mathbf{x}')$ encodes assumptions about the function space, such as smoothness and periodicity. Kernels can be combined to model different components:

$$k_{\text{total}} = k_1 + k_2 + k_{12},$$

where k_1 and k_2 capture the main effects of features x_1 and x_2 , and k_{12} models their interaction.

Orthogonality between components refers to the statistical independence (zero covariance) between them. Two types of orthogonality are considered:

- **Same-Feature Orthogonality:** The interaction term is orthogonal to the main effects with respect to the same feature.

3D Scatter Plot of Original Data

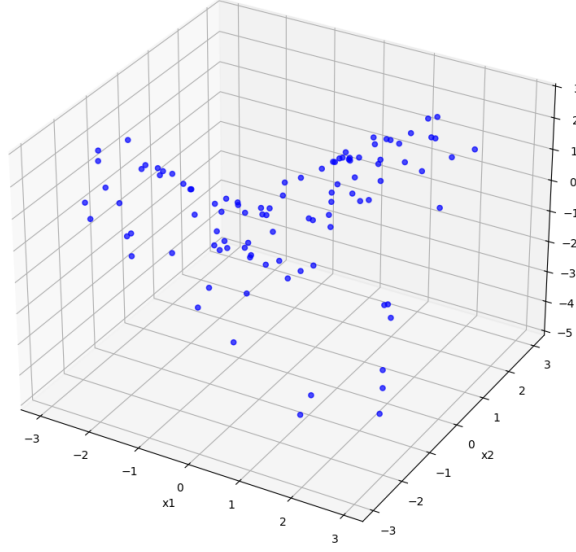


Figure 1: Plot of the synthetic data

- **Cross-Feature Orthogonality:** The interaction term is orthogonal to the main effects with respect to the other feature.

Enforcing orthogonality can enhance interpretability by ensuring that the interaction term captures unique variance not explained by the main effects.

3 Experimental Setup

3.1 Data Generation

We generate synthetic data (shown in fig. 1) to simulate a system with known main effects and interaction:

$$\begin{aligned} x_1 &\sim \text{Uniform}(-3, 3), \\ x_2 &\sim \text{Uniform}(-3, 3), \\ y &= \sin(x_1) + \cos(x_2) + 0.5 \times x_1 x_2 + \varepsilon, \end{aligned}$$

where $\varepsilon \sim \mathcal{N}(0, 0.1^2)$ represents Gaussian noise.

3.2 Modeling with Gaussian Processes

We construct two GP models:

1. **Same-Feature Orthogonality Model:** Uses kernels that ensure the interaction term is orthogonal to the main effects concerning the same feature.
2. **Cross-Feature Orthogonality Model:** Implements kernels designed to make the interaction term orthogonal to the main effects with respect to the other feature.

3.3 Kernel Definitions

3.3.1 Same-Feature Orthogonality Model

The combined kernel is defined as:

$$k_{\text{same}} = k_1 + k_2 + k_{12},$$

where:

$$\begin{aligned} k_1 &= \sigma_1^2 \exp\left(-\frac{(x_1 - x'_1)^2}{2l_1^2}\right), \\ k_2 &= \sigma_2^2 \exp\left(-\frac{(x_2 - x'_2)^2}{2l_2^2}\right), \\ k_{12} &= \sigma_{12}^2 \exp\left(-\frac{(x_1 - x'_1)^2}{2l_{12}^2}\right) \exp\left(-\frac{(x_2 - x'_2)^2}{2l_{12}^2}\right). \end{aligned}$$

3.3.2 Cross-Feature Orthogonality Model

To achieve cross-feature orthogonality, we define the interaction kernel using centered kernels:

$$k_{12}^{\text{cross}} = (k_{x_1} - \mu_{k_{x_1}}) (k_{x_2} - \mu_{k_{x_2}}),$$

where $\mu_{k_{x_i}}$ is the mean of the kernel k_{x_i} over the training data.

4 Implementation Details

4.1 Custom Kernel Class

We implement a custom kernel class for the cross-feature orthogonality model:

```
class CrossFeatureOrthogonalKernel(Kernel):
    def __init__(self, kx1, kx2, X_train):
        self.kx1 = kx1
        self.kx2 = kx2
        self.X_train = X_train
        self.mean_kx1 = None
        self.mean_kx2 = None

    def __call__(self, X, Y=None, eval_gradient=False):
        # Compute centered kernels and their product
        # Raise NotImplementedError if eval_gradient=True
```

Since gradient computation is complex for centered kernels, we disable hyperparameter optimization by setting `optimizer=None` in the GP regressor.

4.2 Model Training

We fit both GP models to the synthetic data:

```
gp_same = GaussianProcessRegressor(kernel=kernel_same, alpha=0.1)
gp_same.fit(X_train, y)

gp_cross = GaussianProcessRegressor(kernel=kernel_cross, alpha=0.1, optimizer=None)
gp_cross.fit(X_train, y)
```

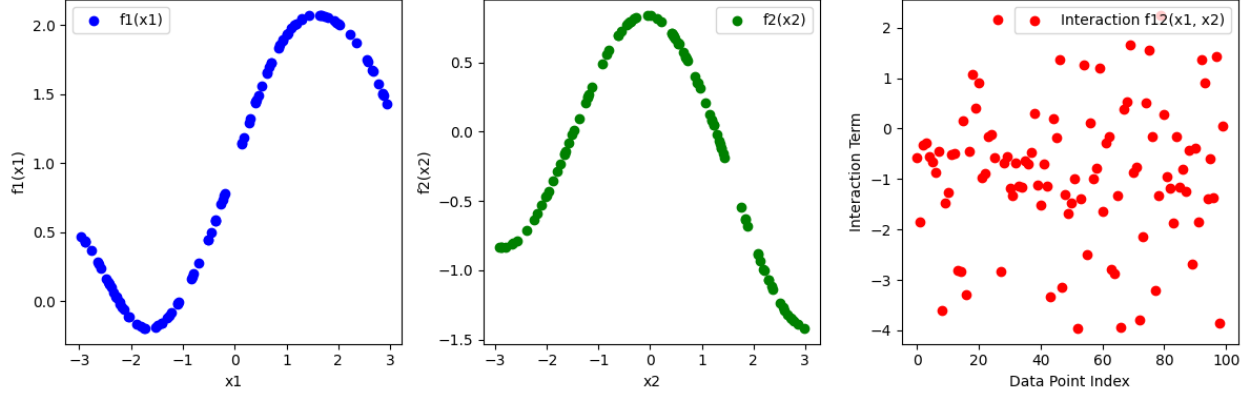


Figure 2: Separated components of each contribution

4.3 Component Decomposition

We decompose the predicted outputs into main effects and interaction terms:

$$\begin{aligned}\hat{f}_1 &= \text{GP}([x_1, \bar{x}_2]), \\ \hat{f}_2 &= \text{GP}([\bar{x}_1, x_2]), \\ \hat{f}_{12} &= \text{GP}([x_1, x_2]) - \hat{f}_1 - \hat{f}_2,\end{aligned}$$

where $\bar{x}_i$ is the mean of x_i .

5 Results

5.1 Covariance and Variance Analysis

We compute the covariance between main effects and interaction terms, as well as the variance of each component. The graphs in figure 2 show the decomposition visually.

5.1.1 Same-Feature Orthogonality Model

- Covariance between f_1 and f_{12} : 0.0274
- Covariance between f_2 and f_{12} : 0.1212
- Variance of f_1 : 0.5062
- Variance of f_2 : 0.4680
- Variance of f_{12} : 1.8634

5.1.2 Cross-Feature Orthogonality Model

- Covariance between f_1 and f_{12} : -0.1029
- Covariance between f_2 and f_{12} : 0.1210
- Variance of f_1 : 0.6988
- Variance of f_2 : 0.4970
- Variance of f_{12} : 1.8623

6 Discussion

6.1 Interpretation of Covariance Results

Covariance between f_1 and f_{12} :

- *Same-Feature Orthogonality:* Small positive covariance (0.0274) suggests a slight dependency between f_1 and f_{12} .
- *Cross-Feature Orthogonality:* Negative covariance (-0.1029) indicates increased independence, as the interaction term is orthogonal to f_1 with respect to x_1 .

Covariance between f_2 and f_{12} :

- Similar values in both models suggest that the cross-feature orthogonality implementation primarily affects the relationship between f_1 and f_{12} .

6.2 Interpretation of Variance Results

Variance of f_1 :

- Increased variance in the cross-feature orthogonality model (from 0.5062 to 0.6988) suggests that f_1 captures more unique variance associated with x_1 .

Variance of f_2 :

- Slight increase indicates a minor change in how f_2 captures variance related to x_2 .

Variance of f_{12} :

- Stable variance across models implies that the interaction term's contribution to explaining data variability remains consistent.

6.3 Implications

- **Enhanced Interpretability:** The cross-feature orthogonality model more effectively separates the main effect f_1 from the interaction term, improving interpretability.
- **Component Independence:** Reduced covariance between f_1 and f_{12} aligns with the goal of achieving statistical independence between components.

7 Future Work

Kernel Refinement: Further refinement of the kernel to balance the covariance reductions between f_1 and f_2 could enhance the model's ability to capture interactions symmetrically.

Gradient Implementation: Implementing gradient computations for the custom kernel would enable hyperparameter optimization, potentially improving model performance.

Performance Evaluation: Evaluating the models using metrics like Mean Squared Error (MSE) and conducting cross-validation would provide insights into their predictive capabilities.

Alternative Kernels: Exploring different kernel functions or combinations may yield better orthogonality and model performance.

8 Conclusion

This experiment demonstrates that enforcing cross-feature orthogonality in Gaussian Process models affects the relationships between main effects and interaction terms. The cross-feature orthogonality model reduces the covariance between f_1 and f_{12} , indicating increased independence and enhanced interpretability. Future work includes refining the kernel implementation, enabling hyperparameter optimization, and thoroughly evaluating model performance.

References

- [1] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.